KOÇ UNIVERSITY

COMP 350
Introduction to DevOps

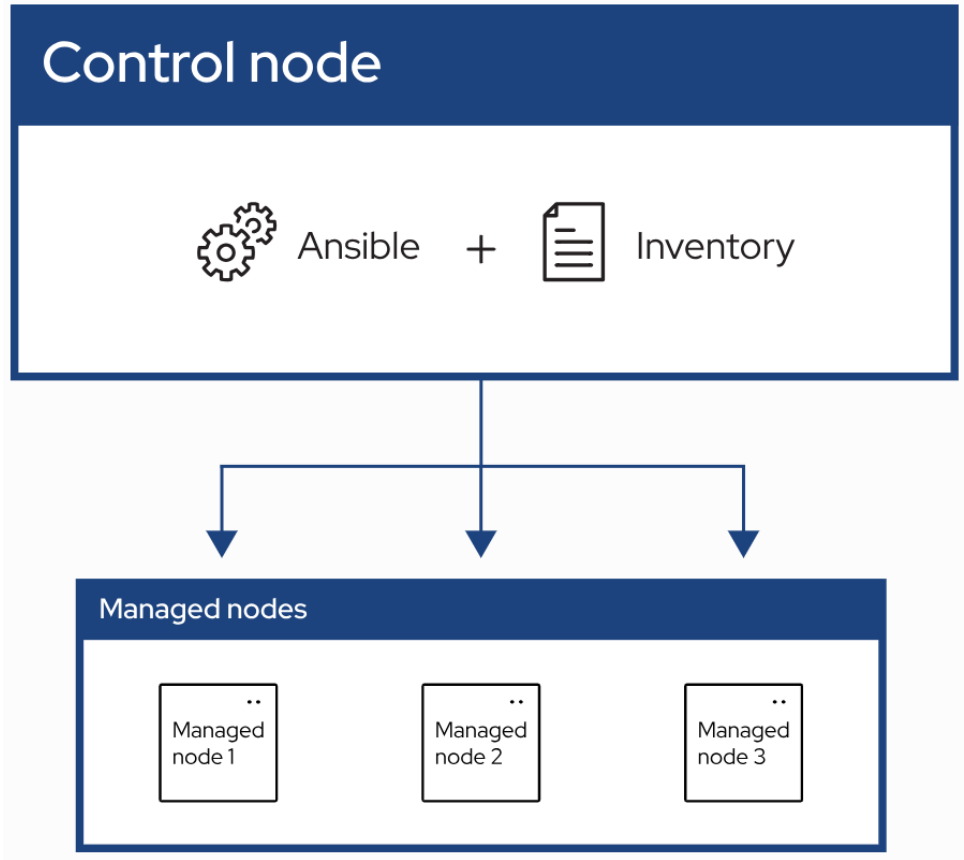
Lecture 12
Ansible (Part 1)

Hakan Ayral

Getting started with Ansible

- Ansible **automates** the management of remote systems and **controls** their desired state.
- Most Ansible environments have three main components:

1. **Control node**
   - A system on which Ansible is installed. You run Ansible commands such as ansible or ansible-inventory on a control node.
2. **Inventory**
   - A list of managed nodes that are logically organized.
   - You create an inventory on the control node to describe host deployments to Ansible.
3. **Managed node**
   - A remote system, or host, that Ansible controls.



Introduction to Ansible

- Open-source automation
  - reduces complexity
  - runs everywhere
  - automate virtually any task
- Use cases for Ansible:
  - Eliminate repetition and simplify workflows
  - Manage and maintain system configuration
  - **CD** (continuous deployment) complex software
  - Perform zero-downtime **rolling updates**
- Use simple, human-readable scripts called **playbooks** to automate your tasks:
  - Declarative: define the desired state of a local or remote system in your playbook
  - Ansible ensures that the system remains in that state.

- Ansible is designed around the following principles:
  - **Agent-less** architecture
    - Low maintenance overhead by avoiding the installation of additional software
  - Simplicity
    - **Playbooks** in YAML
    - **Decentralized**, uses SSH and existing OS credentials to access remote machines.
  - Scalability and flexibility
    - Scale up/down the systems you automate through a modular design
      - supports a large range of OS, cloud platforms, and network devices.
  - **Idempotence** and predictability
    - When the system is in the state your playbook describes Ansible does not change anything, even if the playbook runs multiple times.

Start automating with Ansible

- Get started with Ansible by creating an automation project, building an inventory, and creating a “Hello World” playbook.
- Step1 Install Ansible:

```
pip install ansible
```

- Alternative 1:

```
apt install ansible-core
```

- Alternative 2:

```
apt-add-repository ppa:ansible/ansible -y
apt update
apt install -y aptitude ansible dialog
```

Start automating with Ansible

- Step 2 : Create a project folder on your filesystem.

```
mkdir ansible_quickstart
cd ansible_quickstart
```

- Using a single directory structure makes it easier to add to source control as well as to reuse and share automation content.

Building an **inventory**

- Inventories organize managed nodes in centralized files that provide Ansible with system information and network locations.
  - Using an inventory file, Ansible can manage a large number of hosts with a single command.
- You need the IP addresses or domain names.
  - The host could be a real physical machine, a virtual machine or a container.
  - Ensure that your public SSH key is added to the **authorized_keys** file on each host.
- build an inventory as follows:
  1. Create a file named **inventory.ini** in the directory that you created in the preceding step.
  2. Add a new **[myhosts]** group to the **inventory.ini** file and specify the IP address or domain name of each host system:

```
[myhosts]
192.0.2.50
192.0.2.51
192.0.2.52
```

  3. Verify your inventory: **ansible-inventory -i inventory.ini --list**
  4. Ping the myhosts group in your inventory: **ansible myhosts -m ping -i inventory.ini**

Building an **inventory**

```
ansible myhosts -m ping -i inventory.ini
```

- **-m** for module name
- **-i** for inventory file
- **-u** ← only if the username is different on the control node and the managed node(s).
- Command line above will execute the ping module on every host defined in the inventory file.
  - Ansible ping module is a complex python script, different from the linux ping command which uses ICMP echo packets.

```
192.0.2.50 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
192.0.2.51 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
192.0.2.52 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
```

Inventories in **INI** or **YAML** format

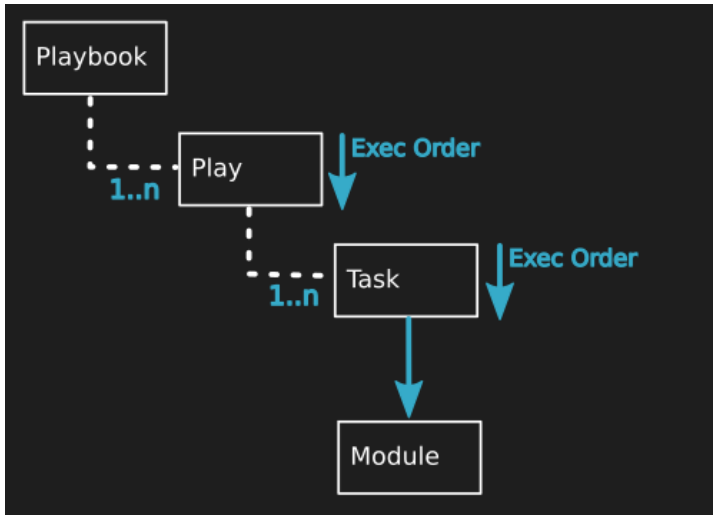
- You can create inventories in either **INI** files or in **YAML**.
- Keeping inventory in **YAML** format makes more sense as the number of managed nodes increases.
- The following is an equivalent of the **inventory.ini** that declares unique names for managed nodes and uses the **ansible_host** field:

```
myhosts:
  hosts:
    my_host_01:
      ansible_host: 192.0.2.50
    my_host_02:
      ansible_host: 192.0.2.51
    my_host_03:
      ansible_host: 192.0.2.52
```

You can name your inventory file just **inventory** without any extension

Creating a **playbook**

- Playbooks are automation blueprints, in YAML format, that Ansible uses to deploy and configure managed nodes.
- Core concepts:
  - **Playbook**
    - A list of **plays** that define the order in which Ansible performs operations, from top to bottom, to achieve an overall goal.
  - **Play**
    - An ordered list of **tasks** that maps to managed nodes in an inventory.
  - **Task**
    - A reference to a single **module** that defines the operations that Ansible performs.
  - **Module**
    - A unit of code or binary that Ansible runs on managed nodes.
    - Ansible modules are grouped in collections with a Fully Qualified Collection Name (FQCN) for each module.



Creating a playbook

- Steps to create a playbook that pings your hosts and prints a “Hello world” message:
  1. Create a file named **playbook.yaml** in your directory, that you created earlier, with the following content:

```yaml
- name: My first play
  hosts: myhosts
  tasks:
   - name: Ping my hosts
     ansible.builtin.ping:

   - name: Print message
     ansible.builtin.debug:
      msg: Hello world
```

  2. Run your playbook:

```
ansible-playbook -i inventory.ini playbook.yaml
```

  3. Ansible returns the following output:

```
PLAY [My first play] ***********************************************************************

TASK [Gathering Facts] *********************************************************************
ok: [192.0.2.50]
ok: [192.0.2.51]
ok: [192.0.2.52]

TASK [Ping my hosts] ***********************************************************************
ok: [192.0.2.50]
ok: [192.0.2.51]
ok: [192.0.2.52]

TASK [Print message] ************************************************************************
ok: [192.0.2.50] => {
    "msg": "Hello world"
}
ok: [192.0.2.51] => {
    "msg": "Hello world"
}
ok: [192.0.2.52] => {
    "msg": "Hello world"
}

PLAY RECAP *********************************************************************************
192.0.2.50: ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
192.0.2.51: ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
192.0.2.52: ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

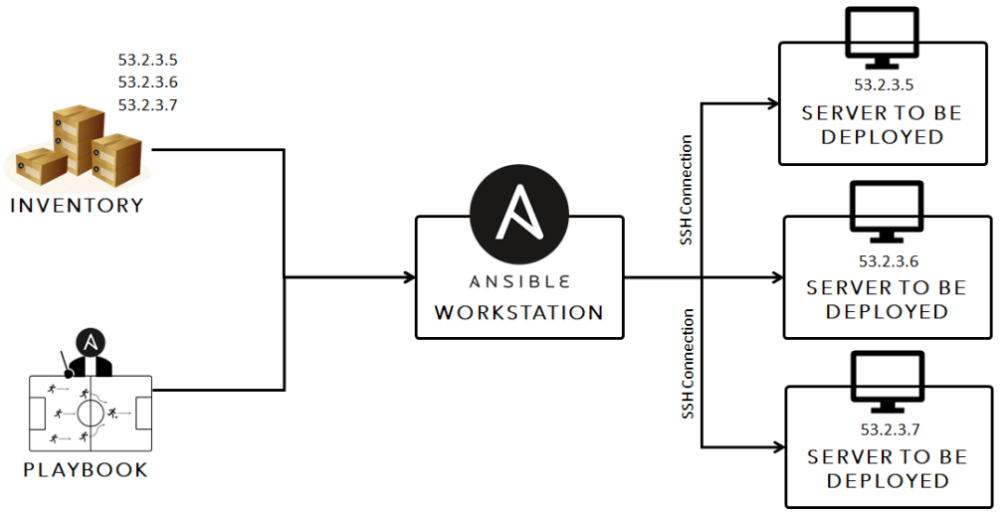
Creating a playbook

- In this output you can see:

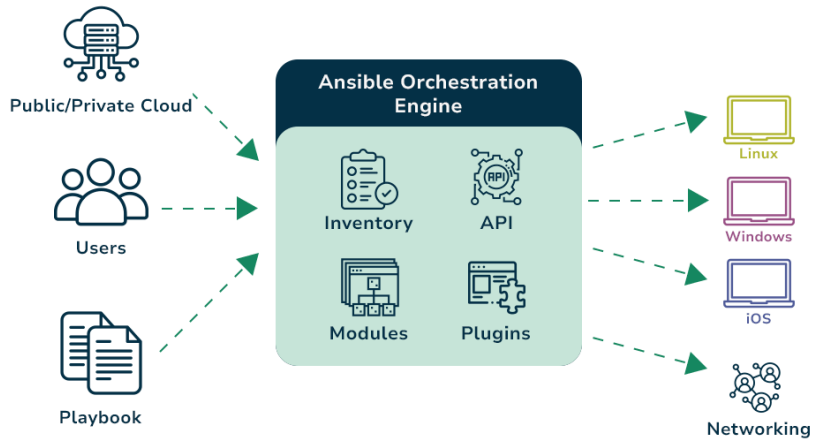
- Names that you give the play and each task.
  - Use descriptive names that make it easy to verify and troubleshoot playbooks.
- `Gathering Facts` task runs implicitly.
  - By default, Ansible gathers information about your inventory.
- Status of each task.
  - Here, each task has a status of **ok** which means it ran successfully.
- The play recap that summarizes results of all tasks in the playbook per host.
  - In this example, there are three tasks so **ok=3** indicates that each task ran successfully.

Ansible concepts

- These concepts are common to all uses of Ansible:
  - Control node
  - Managed nodes
  - Inventory
  - Playbooks
    - Plays
      - Roles
      - Tasks
      - Handlers
  - Modules
  - Plugins
  - Collections



Ansible Architecture Diagram



Control node

- The machine from which you run the Ansible CLI tools (**ansible-playbook** , **ansible**, **ansible-vault** and others).
- You can use any computer that meets the software requirements as a control node - laptops, shared desktops, and servers can all run Ansible.
- You can also run Ansible in containers known as **Execution Environments**.
- Multiple control nodes are possible, but Ansible itself does not coordinate across them, see AAP for such features.

Managed Nodes

- Also referred to as ‘hosts’, these are the target devices (servers, network appliances or any computer) you aim to manage with Ansible.
- Ansible is not normally installed on managed nodes, unless you are using **ansible-pull**, but this is rare and not the recommended setup.

Inventory

- A list of managed nodes provided by one or more ‘inventory sources’.
  - Your inventory can specify information specific to each node, like IP address.
  - It is also used for assigning groups, that both allow for node selection in the Play and bulk variable assignment.
- Sometimes an inventory source file is also referred to as a ‘hostfile’.

Playbooks

- They contain Plays (which are the basic unit of Ansible execution). This is both an ‘execution concept’ and how we describe the files on which **ansible-playbook** operates.
- Playbooks are written in YAML and are easy to read, write, share and understand.
  - **Plays**
    - The main context for Ansible execution, this playbook object maps managed nodes (hosts) to tasks. The Play contains variables, roles and an ordered lists of tasks and can be run repeatedly. It basically consists of an implicit loop over the mapped hosts and tasks and defines how to iterate over them.
  - **Roles**
    - A limited distribution of reusable Ansible content (tasks, handlers, variables, plugins, templates and files) for use inside of a Play.
    - To use any Role resource, the Role itself must be imported into the Play.
  - **Tasks**
    - The definition of an ‘action’ to be applied to the managed host. You can execute a single task once with an ad hoc command using ansible or ansible-console (both create a virtual Play).
  - **Handlers**
    - A special form of a Task, that only executes when notified by a previous task which resulted in a ‘changed’ status.

Setup global key file and inventory file preference (ansible.cfg)

- Normally the part in **blue** specified below is common for all commands:

```
ansible all --key-file ~/.ssh/ansible -i inventory –m <module-name>
```

- You don’t need to specify the key file and inventory file as a parameter everytime; you can set these parameters in the **ansible.cfg** file. Ansible will use these defaults for all further operations without asking:

```
[defaults]
inventory = inventory
private_key_file = ~/.ssh/ansible
```

- There is also another global **ansible.cfg** file in **/etc/ansible** but the local one has precedence.

Check the contens of effective list of hosts

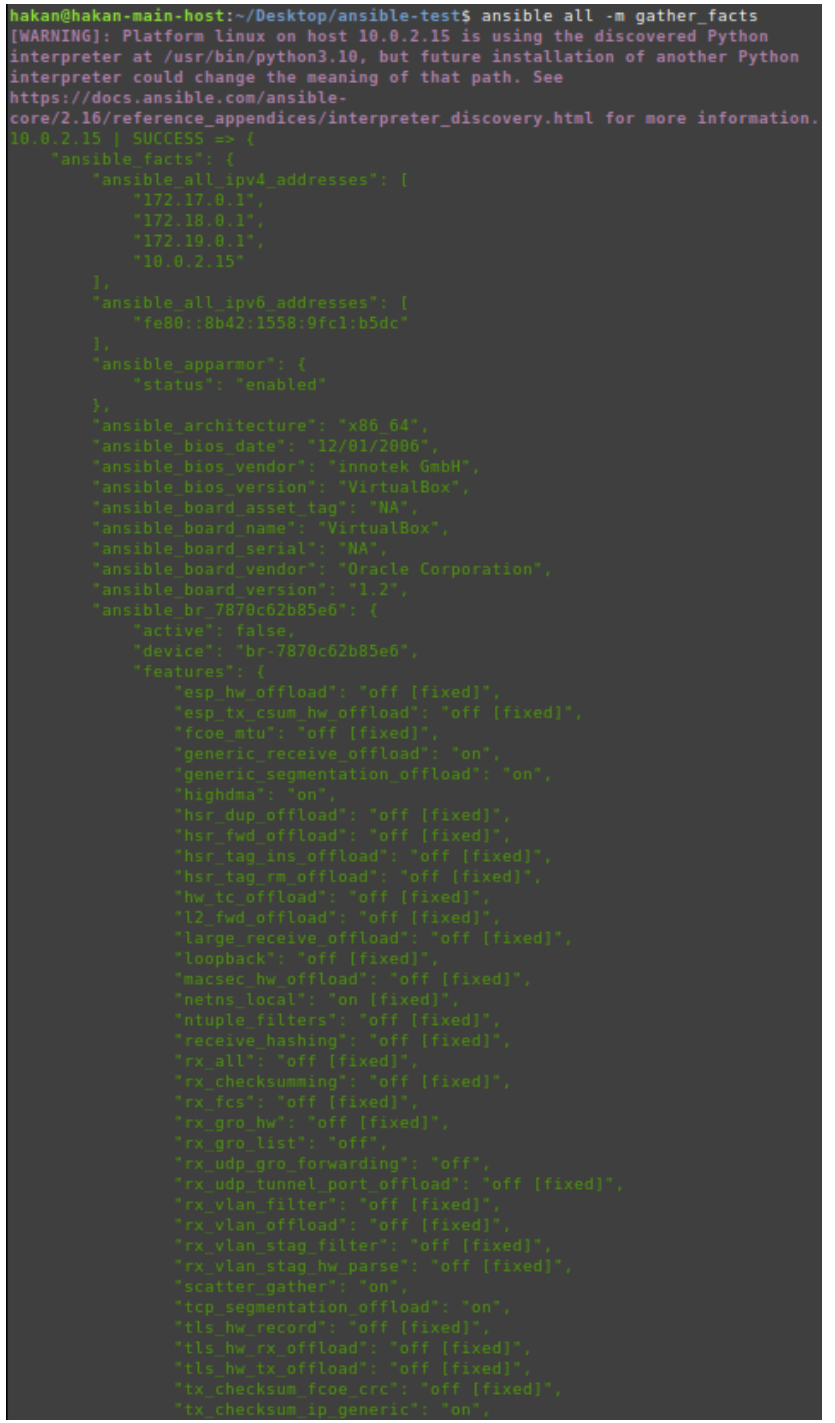
- You can use **ansible all --list-hosts** to see the effective hosts list derived from the global or local ansible.cfg

```
y@workstation:~/ansible_tutorial$ ansible all --list-hosts
hosts (3):
  172.16.250.132
  172.16.250.133
  172.16.250.134
```

Get detailed information about hosts

- You can use **ansible all --m gather_facts** to launch the **gather_facts** module against all hosts

  - There are lots of useful modules like **ping** and **gather_facts**
  - You can check the list of all modules at https://docs.ansible.com/ansible/2.9/modules/list_of_all_modules.html
    - There are literally hundreds of them



Must know Ansible modules

- **Package management**
  - DNF module is dnf_module,
  - old YUM module (required for Python 2 compatibility) is yum_module,
  - APT module is apt_module,
  - Slackpkg module is slackpkg_module

```
- name: install the latest version of Apache and MariaDB
  dnf:
    name:
      - httpd
      - mariadb-server
    state: latest
```

```
- name: Install a list of packages
  yum:
    name:
      - nginx
      - postgresql
      - postgresql-server
    state: present
```

- **Services**
  - service module enables you to start, stop, and reload installed packages

```
- name: Start service foo, based on running process /usr/bin/foo
  service:
    name: foo
    pattern: /usr/bin/foo
    state: started
```

```
- name: Restart network service for interface eth0
  service:
    name: network
    state: restarted
    args: eth0
```

Must know Ansible modules

- **Copy files**
  - copy module copies a file from the local or remote machine to a location on the remote machine.

```
- name: Copy a new "ntp.conf file into place, backing up the original if it diffe
  copy:
    src: /mine/ntp.conf
    dest: /etc/ntp.conf
    owner: root
    group: root
    mode: '0644'
    backup: yes
```

```
- name: Copy file with owner and permission, using symbolic representation
  copy:
    src: /srv/myfiles/foo.conf
    dest: /etc/foo.conf
    owner: foo
    group: foo
    mode: u=rw,g=r,o=r
```

- **Debug**
  - debug module prints statements during execution and can be useful for debugging variables or expressions without having to halt the playbook.

```
- name: Display all variables/facts known for a host
  debug:
    var: hostvars[inventory_hostname]
    verbosity: 4
```

```
- name: Write some content in a file /tmp/foo.txt
  copy:
    dest: /tmp/foo.txt
    content: |
      Good Morning!
      Awesome sunshine today.
    register: display_file_content
- name: Debug display_file_content
    debug:
      var: display_file_content
      verbosity: 2
```

```
ansible-playbook demo.yaml -vv
```

Must know Ansible modules

- **File:** file module manages the file and its properties.
  - It sets attributes of files, symlinks, or directories.
  - It also removes files, symlinks, or directories.

```
- name: Change file ownership, group and permissions
  file:
    path: /etc/foo.conf
    owner: foo
    group: foo
    mode: '0644'
```

```
- name: Create a directory if it does not exist
  file:
    path: /etc/some_directory
    state: directory
    mode: '0755'
```

- **Lineinfile:** lineinfile module manages lines in a text file.
  - Ensures a particular line is in a file or replaces an existing line using a back-referenced regular expression.
  - Useful when you want to change just a single line in a file.

```
- name: Ensure SELinux is set to enforcing mode
  lineinfile:
    path: /etc/selinux/config
    regexp: '^SELINUX='
    line: SELINUX=enforcing
```

```
- name: Add a line to a file if the file does not exist, without passing regexp
  lineinfile:
    path: /etc/resolv.conf
    line: 192.168.1.99 foo.lab.net foo
    create: yes
```

Must know Ansible modules

- **Git:** git module manages git checkouts of repositories to deploy files or software.

```
# Example Create git archive from repo
- git:
    repo: https://github.com/ansible/ansible-examples.git
    dest: /src/ansible-examples
    archive: /tmp/ansible-examples.zip
```

```
- git:
    repo: https://github.com/ansible/ansible-examples.git
    dest: /src/ansible-examples
    separate_git_dir: /src/ansible-examples.git
```

- **Archive:** archive module creates a compressed archive of one or more files.
  - it assumes the compression source exists on the target.

```
- name: Compress directory /path/to/foo/ into /path/to/foo.tgz
  archive:
    path: /path/to/foo
    dest: /path/to/foo.tgz
```

```
- name: Create a bz2 archive of multiple files, rooted at /path
  archive:
    path:
    - /path/to/foo
    - /path/wong/foo
    dest: /path/file.tar.bz2
    format: bz2
```

Must know Ansible modules

- **Command:** command module takes the command name followed by a list of space-delimited arguments.

Example 1:

```
- name: return motd to registered var
  command: cat /etc/motd
  register: mymotd
```

Example 2:

```
- name: Change the working directory to somedir/ and run the command as db_owner
  command: /usr/bin/make_database.sh db_user db_name
  become: yes
  become_user: db_owner
  args:
    chdir: somedir/
    creates: /path/to/database
```

Run **apt** module on remote through ansible

- You can use

ansible all –m **apt** –a update_cache=true

```
hakan@hakan-main-host:~/Desktop/ansible-test$ ansible all -m apt -a update_cache=true
[WARNING]: Platform linux on host 10.0.2.15 is using the discovered Python interpreter at /usr/bin/python3.10, but
future installation of another Python interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for more information.
10.0.2.15 | FAILED! => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.10"
    },
    "changed": false,
    "msg": "Failed to lock apt for exclusive operation: Failed to lock directory /var/lib/apt/lists/: E:Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)"
}
```

Doesn't work, needs root

ansible all –m **apt** –a update_cache=true **--become --ask-become-pass** ← **This solves the problem**

```
hakan@hakan-main-host:~/Desktop/ansible-test$ ansible all -m apt -a update_cache=true --become --ask-become-pass
BECOME password:
[WARNING]: Platform linux on host 10.0.2.15 is using the discovered Python interpreter at /usr/bin/python3.10, but future installation of another Python interpreter could
change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for more information.
10.0.2.15 | CHANGED => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.10"
    },
    "cache_update_time": 1716199526,
    "cache_updated": true,
    "changed": true
}
hakan@hakan-main-host:~/Desktop/ansible-test$
```

Install software through **apt** module

- Use

```
ansible all –m apt –a name=tmux --become --ask-become-pass
```

- **apt** is just one of hundreds of modules in Ansible
  - See documentation for apt module here https://docs.ansible.com/ansible/2.9/modules/apt_module.html#apt-module
- We can use this to create our first **playbook**!
  - A playbook is a text file in yaml format, which contains a list of multiple **plays**, where each play in turn contains multiple **tasks**.

Create a playbook

```
 GNU nano 4.8                                  install_apache.yml
---

- hosts: all
  become: true
  tasks:

  - name: install apache2 package
    apt:
      name: apache2
```

- You can run this playbook by

```
ansible-playbook --ask-become-pass install-apache.yml
```

- When **--ask-become-pass** is used user will need to MANUALLY enter password every time
- You can avoid it by adjusting your playbook and your inventory files.

A playbook with multiple taks

```
 GNU nano 4.8                                  install_apache.yml
---

- hosts: all
  become: true
  tasks:

  - name: update repository index
    apt:
      update_cache: yes

  - name: install apache2 package
    apt:
      name: apache2

  - name: add php support for apache
    apt:
      name: libapache2-mod-php
```

- It only makes sense to have a playbook if you have multiple tasks in a play; otherwise you can launch the single command from CLI.

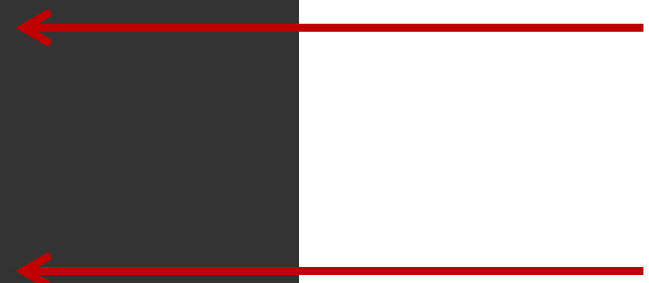
A playbook with conditionals

```
 GNU nano 4.8                                  install_apache.yml
---

- hosts: all
  become: true
  tasks:

  - name: update repository index
    apt:
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

  - name: install apache2 package
    apt:
      name: apache2
      state: latest
    when: ansible_distribution == "Ubuntu"
```



- You can add multiple options to condition as follows:

```
when: ansible_distribution in ['Debian','Ubuntu']
```

- You can also use and / or between multiple conditions like this **when: (.... and ...) and (... or ...)**